

Sorting

Sean

May 14, 2026

Outline

- Why sorting helps
- Using `sort`
- Custom ordering
- Common sorting patterns



Introduction

- Sorting puts data into a useful order
- Once data is ordered, many repeated comparisons become unnecessary
- In competitive programming, sorting is often the first step that turns a brute-force idea into an efficient one

1 Why Sorting Helps

2 Using sort

3 Custom Ordering

4 Common Sorting Patterns



Why Sorting Helps

Why Sorting Helps

- Sorting groups related values together
- It reveals smallest, largest, nearest, and duplicate values
- It lets us process elements in a meaningful order
- It often changes a problem from “compare everything with everything” to “scan once after sorting”



Sorting Complexity

- `C++ sort` runs in $\mathcal{O}(n \log n)$ time
- This is usually fast enough for $n \leq 2 \times 10^5$
- Sorting can be more expensive than one linear scan, but much cheaper than checking all pairs

$$\mathcal{O}(n^2) \implies \mathcal{O}(n \log n + n)$$



Simple Sorts

- Bubble sort, selection sort, and insertion sort are common beginner sorting algorithms
- They usually take $\mathcal{O}(n^2)$ time
- That is fine for small inputs, but too slow for many CP constraints
- In competitive programming, we usually use `sort` unless a problem needs something more specialized



Example: Duplicates

Task

Given n numbers, decide whether any value appears at least twice.

- Brute force: compare every pair, $\mathcal{O}(n^2)$
- After sorting, equal values become adjacent
- Scan neighboring pairs only, $\mathcal{O}(n \log n)$ total



1 Why Sorting Helps

2 Using sort

3 Custom Ordering

4 Common Sorting Patterns

Using sort

Sorting a Vector

```
1 vector<int> a = {5, 1, 4, 2, 3};  
2 sort(a.begin(), a.end());
```

- `begin()` points to the first element
- `end()` points one past the last element
- The default order is increasing



Decreasing Order

```
1 sort(a.begin(), a.end(), greater<int>());
```

- The third argument tells `sort` how to compare elements
- `greater<int>()` gives decreasing order for integers
- This is useful when we want to process largest values first



Sorting Part of an Array

```
1  sort(a.begin() + l, a.begin() + r + 1);
```

- This sorts the subarray $[l, r]$ when using 0-based indexing
- The right endpoint passed to `sort` is exclusive
- Be careful with off-by-one errors



1 Why Sorting Helps

2 Using sort

3 Custom Ordering

4 Common Sorting Patterns



Custom Ordering

Sorting Pairs

```
1 vector<pair<int, int>> points;  
2 sort(points.begin(), points.end());
```

- Pairs are sorted by the first value
- If the first values tie, they are sorted by the second value
- This is called lexicographic order



Custom Comparator

```
1 sort(v.begin(), v.end(), [](const Item& a,  
2                             const Item& b) {  
3     return a.score > b.score;  
4 });
```

- Return true if a should come before b
- This example sorts by decreasing score
- Comparators should describe a consistent order



Tie-Breaking

```
1 sort(v.begin(), v.end(), [](const Item& a,  
2                             const Item& b) {  
3     if (a.score != b.score) return a.score > b.score;  
4     return a.name < b.name;  
5 });
```

- First sort by decreasing score
- If scores are equal, sort by increasing name
- Tie-breaking makes the order predictable



Capturing Outside Data

```
1 sort(v.begin(), v.end(), [&](int a, int b) {  
2     return pos[a] < pos[b];  
3 });
```

- Use `[&]` if the comparator needs variables or containers from outside the lambda
- Here the comparator reads from `pos`
- Keep the comparison rule consistent even when using outside data



Comparator Mistakes

- Do not use `<=` instead of `<` in a comparator
- Do not make the answer depend on changing outside state
- If two elements are equal by your rule, both `cmp(a, b)` and `cmp(b, a)` should be false
- A bad comparator can make `sort` behave unpredictably



1 Why Sorting Helps

2 Using sort

3 Custom Ordering

4 Common Sorting Patterns



Common Sorting Patterns

Pattern: Sort Then Scan

- Sort the data
- Scan from left to right
- Keep only the information needed from previous elements
- This often gives $\mathcal{O}(n \log n)$ time



Example: Apartments

CSES: Apartments

Applicants have desired apartment sizes. Apartments have actual sizes. An applicant accepts an apartment if the size differs by at most k . Maximize the number of matches.

- Sort applicants and apartments by size
- Match the smallest possible applicant with the smallest possible apartment
- Move one pointer or both pointers depending on the comparison



Pattern: Sort Events

- Convert each important moment into an event
- Sort events by position or time
- Decide how ties are handled before scanning
- Process events in order while maintaining the current state
- This is the beginning of sweep-line thinking



Example: Restaurant Customers

CSES: Restaurant Customers

Given arrival and leaving times of customers, find the maximum number of customers present at the same time.

- Create a $+1$ event for each arrival
- Create a -1 event for each leaving time
- If times can tie, process leaving events before arrivals
- Sort events by time and scan them in order



What Sorting Does Not Solve

- Sorting changes the order of elements
- If original positions matter, store the index before sorting
- If the data changes after every query, sorting once may not be enough
- Sorting is a tool, not a proof of correctness by itself



Takeaways

- Sorting costs $\mathcal{O}(n \log n)$
- Use sorting to reveal order, adjacency, duplicates, and extremes
- Learn the default order of pairs and how to write comparators
- Many CP solutions are: sort, then scan carefully



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

